Design Patterns

Introduction to Design Patterns

- ✓ **Practical proven solutions for recurring software design problems.**
- ✓ **Guide software structure and improve flexibility, reusability, and maintainability.**
- ✓ **Not a code library;** a conceptual design solution applied by developers.
- ✓ Similar to tested recipes for solving common design challenges.

Design patterns are practical, proven solutions for recurring software design problems. They are not code libraries or finished code—they are conceptual templates that developers adapt to solve common design challenges.

Importance and Applications of Design Patterns

- ✓ **Reduce repeated design effort and improve software quality.**
- ✓ Create reusable and maintainable object-oriented systems.
- ✓ **Provide common communication language** among developers.
- ✓ Applications: enterprise systems, web applications, frameworks, large software projects.

Gang of Four (GoF) Pattern Catalogue

- Introduced by Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides.
- **Contains 23 classic object-oriented design patterns.**
- **Patterns are grouped based on purpose**: **Creational, Structural, Behavioural**.
- Provides a common reference for solving software design problems.

Pattern Languages

- A collection of related patterns for a specific problem domain.
- Pattern selection **depends on system context** and design goals.
- Different **domains require different combinations** of patterns.

Categories of Design Patterns

- ✓ Creational Patterns: object creation and initialization.
- ✓ Structural Patterns: organization and relationships among objects.
- ✓ Behavioural Patterns: communication and responsibility distribution.

Gang of Four (GoF) Pattern Catalogue and Categories of Design Patterns

- **Gang of Four (GoF):** The book "Design Patterns: Elements of Reusable Object-Oriented Software" was published in 1994 by Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides.
- **Purpose:** To describe 23 proven solutions (patterns) to common object-oriented design problems. These patterns promote reusable, flexible, and maintainable software design.

The Gang of Four (GoF) Authors


Erich Gamma


Richard Helm


Ralph Johnson


John Vlissides

23 Design Patterns in GoF Catalogue

Creational (5)

1. Singleton
2. Factory Method
3. Abstract Factory
4. Builder
5. Prototype

Structural (7)

1. Adapter
2. Bridge
3. Composite
4. Decorator
5. Facade
6. Flyweight
7. Proxy

Behavioural (11)

1. Template Method
2. Chain of Responsibility
3. Command
4. Interpreter
5. Iterator
6. Mediator
7. Memento
8. Observer
9. State
10. Strategy
11. Visitor

Three Categories of Design Patterns

1. Creational Patterns

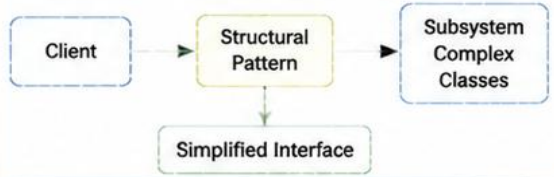
Focus: Object creation mechanisms.
Goal: Make a system independent of how its objects are created, composed, and represented.



Examples: Singleton, Factory Method, Abstract Factory, Builder, Prototype

2. Structural Patterns

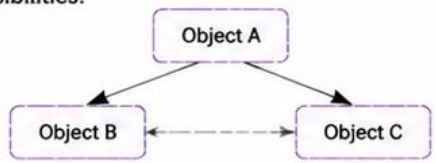
Focus: Class and object composition.
Goal: Build larger structures from simpler components and expose easy-to-use interfaces.



Examples: Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy

3. Behavioural Patterns

Focus: Communication between objects.
Goal: Define how objects interact and distribute responsibilities.



Examples: Template Method, Chain of Responsibility, Command, Iterator, Observer, State, Strategy, Visitor, Mediator, Memento, Interpreter

Design Patterns are time-tested solutions to recurring design problems. They help us write software that is **reusable**, **flexible**, **maintainable**, and easy to understand.

Creational Patterns

- ✓ Focus on flexible object creation mechanisms.
- ✓ Reduce dependency on concrete object classes.
- ✓ **Examples:** Singleton Pattern, Factory Method Pattern.

Make a system independent of how its objects are created, composed, and represented.

Singleton Pattern

- Creates only one instance of a class.
- Useful when a single shared object is required.
- **Implementation:** private constructor, unique instance variable, getInstance() method.
- **Examples:** database connection, printer queue, configuration manager.

Ensures a class has only one instance and provides a global point of access to it.

Singleton Pattern

1. Create private static instance variable.
2. Hide constructor using private access modifier.
3. Create public method to provide instance access.
4. Return existing instance if already created.

Factory Method Pattern

- Separates object creation from object usage.
- Client depends on general interfaces instead of concrete classes.
- Allows new object types without modifying client code.

Factory Method Steps

1. Create Product interface or abstract class.
2. Create concrete product classes.
3. Define factory method in creator class.
4. Subclasses decide which object to create.

Singleton Pattern vs Factory Method Pattern

SINGLETON PATTERN

Concept

Singleton pattern ensures that a class has only one instance and provides a global point of access to it.
It controls object creation and is used when exactly one object is needed to coordinate actions across the system.

When to Use

- When exactly one instance of a class is required.
- To control access to shared resource (e.g., configuration, logger, cache, database connection).

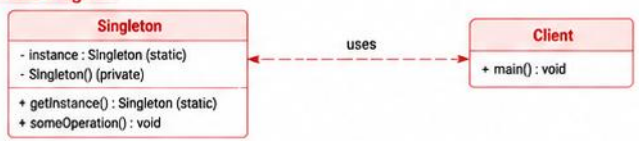
Real-life Examples

- Logger in an application (writes logs to a single file).
- Database connection manager.
- Configuration or settings manager.

Key Benefits

- Controlled access to the single instance.
- Reduced memory usage.
- Global access point.
- Ensures consistency of state.

UML Class Diagram



How it Works



1. Client calls getInstance().
2. If instance is null, create a new one.
3. Store the instance and return it.
4. Next time, return the same instance.

Java Code Example

Singleton Class

```
public class Logger {
   // 1. private static instance
   private static Logger instance;

   // 2. private constructor
   private Logger() {
      System.out.println("Logger instance created");
   }
   // 3. public static method to get the instance
   public static Logger getInstance() {
      if (instance == null) {
          instance = new Logger();
      }
      return instance;
   }
   public void log(String message) {
      System.out.println("LOG: " + message);
   }
}
```

Client Class

```
public class CIient {
   public static void main(String[] args) {
      Logger logger1 = Logger.getInstance();
      logger1.log("First log");

      Logger logger2 = Logger.getInstance();
      logger2.log("Second log");

      System.out.println("Same instance? " +
         (logger1 == logger2));  // true
   }
}
```

Output:
Logger instance created
LOG: First log
LOG: Second log
Same instance? true

Key Point: The constructor is private and the instance is static. Therefore, only one instance can be created and shared.

FACTORY METHOD PATTERN

Concept

Factory Method pattern defines an interface for creating an object, but lets subclasses decide which class to instantiate.
It delegates object creation to subclasses.

When to Use

- When a class can't anticipate the class of objects it must create.
- When subclasses should decide which object to create.
- To achieve loose coupling.

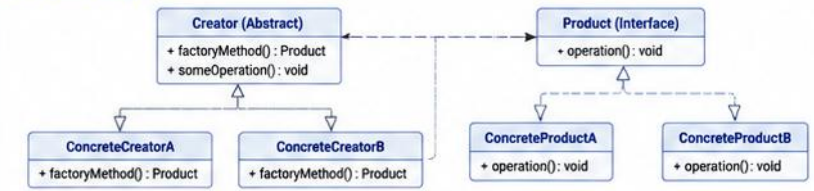
Real-life Examples

- Document creator: Create PDFDocument or WordDocument without client knowing details.
- Notification system: EmailNotification or SMSNotification.
- UI component: Create WindowsButton or MacButton.

Key Benefits

- Promotes loose coupling.
- Easy to add new products without changing client code.
- Follows Open/Closed Principle.

UML Class Diagram



How it Works



1. Client calls someOperation() on Creator.
2. Creator calls factoryMethod().
3. Subclass decides which ConcreteProduct to create.
4. The created product is returned and used by client.

Java Code Example

Product Interface

```
public interface Document {
  void open();
}
```

Concrete Products

```
public class PDFDocument
implements Document {
  public void open() {
    System.out.println("Open PDF Document");
  }
}
public class WordDocument
implements Document {
  public void open() {
    System.out.println("Open Word Document");
  }
}
```

Creator (Abstract Class)

```
public abstract class
DocumentCreetor {
  public abstract Document
    factoryMethod();

  public void someOperation() {
    Document doc = factoryMethod();
    doc.open();
  }
}
```

Concrete Creators

```
public class PDFCreator
extends DocumentCreator {
  public Document factoryMethod() {
    return new PDFDocument();
  }
}


public class WordCreator
extends DocumentCreator {
  public Document factoryMethod() {
    return new WordDocument();
  }
}
```

Client Class

```
public class Client {
  public static void main
  (String[] args) {
    DocumentCreator creator1 =
      new PDFCreator();
    creator1.someOperation();
    DocumentCreator creator2 =
      new WordCreator();
    creator2.someOperation();
  }
}
```

Output:
Open PDF Document
Open Word Document

Key Point: The client depends on the Creator abstract class. Subclasses decide which Product implementation to create.

Singleton Pattern vs Factory Method Pattern

Concept, Example, UML and Code

SINGLETON PATTERN

class has only one instance and provide a global

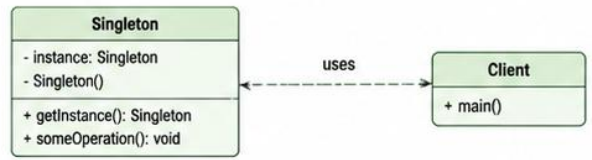
When to Use

- When exactly one instance of a class is needed.
- To control access to shared resources.
- Example: Configuration, Logger, Database Connection.

Key Benefit

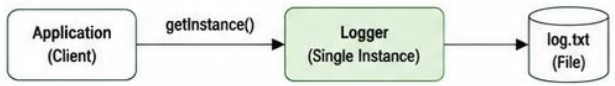
Controlled access to the single instance, reduces memory usage.

UML Class Diagram



Real-life Example

A Logger in an application should have only one instance to write logs to a single file.



Java Code Example

```java
public class Logger {
    private static Logger instance;   // single instance
    private Logger() {
        System.out.println("Logger instance created");
    }
    public static Logger getInstance() {
        if (instance == null) {
            instance = new Logger();
        }
        return instance;
    }
    public void log(String message) {
        System.out.println("LOG: " + message);
    }
}
```

```java
class Client {
    public static void main(String[] args) {
        Logger logger1 = Logger.getInstance();
        logger1.log("First log");

        Logger logger2 = Logger.getInstance();
        logger2.log("Second log");

        System.out.println(logger1 == logger2);
        // Output: true (same instance)
    }
}
```

How it Works

1. Client requests instance
2. If instance is null, create new one
3. Return the same instance to client
4. All clients share the single instance

FACTORY METHOD PATTERN

Define an interface for creating an object, but let subcl… it decide which class to instantiate.

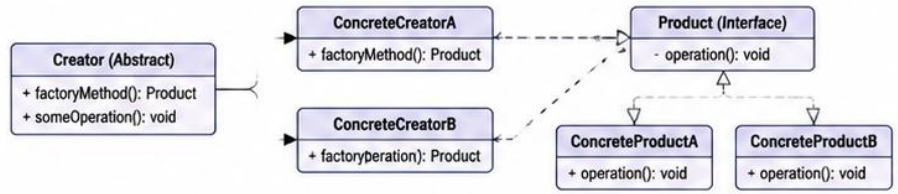
When to Use

- When a class can't anticipate the class of objects it must create.
- When subclasses should decide which object to create.
- Example: Document, Notification, Button in a GUI.

Key Benefit

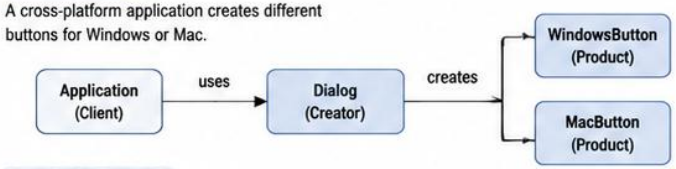
Promotes loose coupling and easy extension (Open/Closed Principle).

UML Class Diagram



Real-life Example

A cross-platform application creates different buttons for Windows or Mac.



Possible Products

- WindowsButton
- MacButton
- LinuxButton
- WebButton

Java Code Example

```java
// Product
interface Button {
  void render();
}

// Concrete Products
class WindowsButton implements Button {
  public void render() {
    System.out.println("Render Windows Button");
  }
}

class MacButton implements Button {
  public void render() {
    System.out.println("Render Mac Button");
  }
}
```

```java
// Creator
abstract class Dialog {
  // Factory Method
  abstract Button createButton();

  public void renderUI() {
    Button b = createButton();
    b.render();
  }
}

// Concrete Creators
class WindowsDialog extends Bialog {
  Button createButton() {
    return new WindowsButton();
  }
```

```java
class MacBialog extends Bialog {
  Button createButton() {
    return new MacButton();
  }
}

// Client
class Client {
  public static void main(String[] args) {
    Bialog dialog;

    dialog = new WindowsDialog();
    dialog.renderUI();   // Render Windows Button

    dialog = new MacDialag();
    dialog.renderUI();   // Render Mac Button
```

How it Works

1. Client calls operation on Creator
2. Creator calls factoryMethod()
3. ConcreteCreator decides which Product to create
4. Product is returned and used by Client

Abstract Factory Pattern, Builder Pattern & Prototype Pattern

Concept, Example, UML Diagram & Java Code

1. ABSTRACT FACTORY PATTERN

Provides an interface for creating *families* of related or dependent objects without specifying their concrete classes.

When to Use

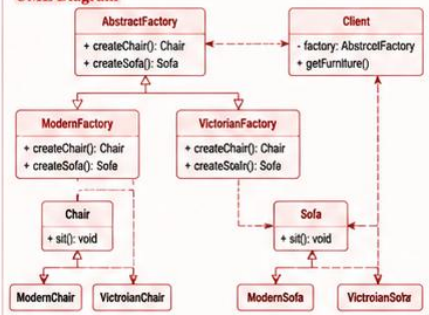
- When a system should be independent of how its products are created, composed and represented.
- When the system must be configured with one of multiple families of products.

Example



A furniture store offers modern and victorian style furniture. Client code uses a factory to get matching chair and sofa.

UML Diagram



Java Code Example

```
// Product Interfaces
interface Chair { void sit(); }
interface Sofa { void sit(); }

// Concrete Products
class ModernChair implements Chair {
    public void sit() {
        System.out.println("Sitting on Modern Chair");
    }
}
class ModernSofa implements Sofa {
    public void sit() {
        System.out.println("Sitting on Modern Sofa");
    }
}
// Abstract Factory
interface FurnitureFactory {
    Chair createChair();
    Sofa createSofa();
}
```

```
// Concrete Factories
class ModernFactory implements FurnitureFactory {
    public Chair createChair() {
        return new ModernChair();
    }
    public Sofa createSofa() {
        System new ModernSofa();
    }
}
// Client
class Client {
    private FurnitureFactory factory;
    Client(FurnitureFactory factory) {
        this.factory = factory;
    }
    void getFurniture() {
        Chair chair = factory.createChair();
        Sofa sofa = factory.createSofa();
        chair.sit();
        sofa.sit();
    }
}
```

How it Works



1. Client selects a concrete factory
2. Factory creates related products
3. Products are used together
4. System remains consistent

2. BUILDER PATTERN

Separates the construction of a complex object from its representation so that the same construction process can create different representations.

When to Use

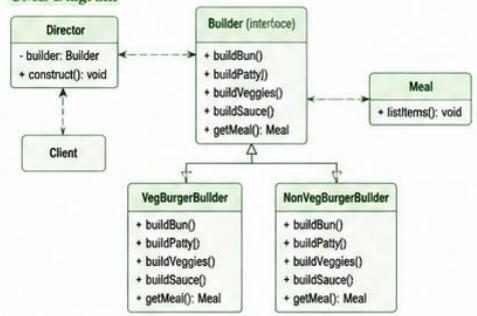
- When the construction process must allow different representations for the object that is constructed.
- When a complex object should be created step by step.

Example



A meal can be a veg or non-veg burger. We build the burger step by step using the same process but with different ingredients.

UML Diagram



Java Code Example

```
// Product
class Meal {
    private List<String> items = new ArrayList<>();
    public void addItem(String item) { items.add(item); }
    public void listItems() {
        System.out.println(items);
    }
}

// Builder
interface Builder {
    void buildBun();
    void buildPatty();
    void buildVeggies();
    void buildSauce();
    Meal getMeal();
}

// Concrete Builder
class VegBurgerBuilder implements Builder {
    private Meal meal = new Meal();
```

```
    public void buildBun() {
        meal.addItem("Veg Bun");
    }
    public void buildPatty() {
        meal.addItem("Aloo Patty");
    }
    public void buildVeggies() {
        meal.addItem("Lettuce");
    }
    public void buildSauce() {
        meal.addItem("Tomato Sauce");
    }
    public Meal getMeal() { return meal; }
        return meal;
    }
}

// Director
class Director {
    private Builder builder;
    Director(Builder builder) {
                { this.builder = builder; }
    }
    void construct() {
        builder.buildBun();
        builder.buildPatty();
        builder.buildVeggies();
        builder.buildSauce();
    }
}
```

How it Works



1. Client chooses a builder
2. Director controls construction steps
3. Builder builds the complex object
4. Final object is returned to client

3. PROTOTYPE PATTERN

Specifies the kinds of objects to create using a prototypical instance, and creates new objects by copying this prototype.

When to Use

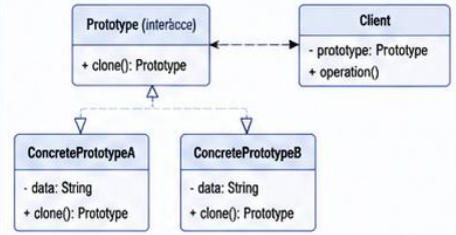
- When the classes to instantiate are specified at runtime.
- To avoid building a class hierarchy of factores that parallels the class hierarchy of products.

Example



A document editor can clone existing documents (letter, report) to create new ones with the same content.

UML Diagram



Java Code Example

```
// Prototype Interface
interface Document extends Cloneable {
    Document clone();
    void showData();
}
// Concrete Prototype
class Letter implements Document {
    private String content;
    public Letter(String content) { this.content = content; }
    public Document clone() {
        return new Letter(this.content);
    }
    public void showData() {
        System.out.println("Letter: " + content);
    }
}
// Client
class Client {
    public static void main(String[] args) {
        Document d1 = new Letter("Hello, this is a letter.");
        Document d2 = d1.clone();  // cloning
        d2.showData();             // Letter: Hello, this is a letter.
    }
}
```

How it Works



1. Client has a prototype object
2. Clone() is called to copy the prototype
3. New object is created
4. Client uses the cloned object

Abstract Factory Pattern, Builder Pattern & Prototype Pattern

Concept, Example, UML Diagram, Java Code, How it Works & Key Benefits

1. ASTRACT FACTORY PATTERN

Concept: Provides an interface for creating families of related or dependent objects without specifying their concrete classes.

When to Use

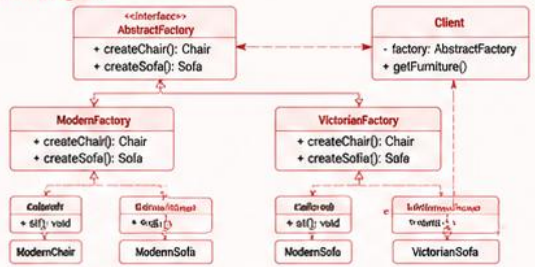
- When a system should be independent of how its products are created, composed and represented.
- When the system must be configured with one of multiple families of products.

Example



A furniture store offers modern and victorian style furniture. Client code uses a factory to get matching chair and sofa.

UML Diagram



Java Code Example

```
// Product Interfaces
interface Chair {
    void sit();
}
interface Sofa {
    void sit();
}
// Concrete Products (Modern)
class ModernChair implements Chair {
    public void sit() {
        System.out.println("Sitting on Modern Chair");
    }
}
class ModernSofa implements Sofa {
    public void sit() {
        System.out.println("Sitting on Modern Sofa");
    }
}
```

```
// Concrete Products (Victorian)
class VictorianChair implements Chair {
    public void sit() {
        System.out.println("Sitting on Victorian Chair");
    }
}

class VictorianSofa implements Sofa {
    public void sit() {
        System.out.println("Sitting on Victorian Sofa");
    }
}
```

```
// Abstract Factory
interface AbstractFactory {
    Chair createChair();
    Sofa createSofa();
}
// Concrete Factories
class ModernFactory implements AbstractFactory {
    public Chair createChair() { return new ModernChair(); }
    public Sofa createSofa() { return new ModernSofa(); }
}
class VictorianFactory implements AbstractFactory {
    public Chair createChair() { return new VictorianChair(); }
    public Sofa createSofa() { return new VictorianSofa(); }
}
// Client
class Client {
    private AbstractFactory factory;
    public Client(AbstractFactory factory) {
        this.factory = factory;
    }
    public void getFurniture() {
        Chair chair = factory.createChair();
        Sofa sofa = factory.createSofa();
        chair.sit(); sofa.sit();
    }
}
```

How it Works



Key Benefits

- ✓ Ensures consistency among related products.
- ✓ Promotes loose coupling.
- ✓ Easy to add new product families.

Trade-off

- Adding a new product type requires changing all factories.

2. BUILDER PATTERN

Concept: Separates the construction of a complex object from its representation so that the same construction process can create different representations.

When to Use

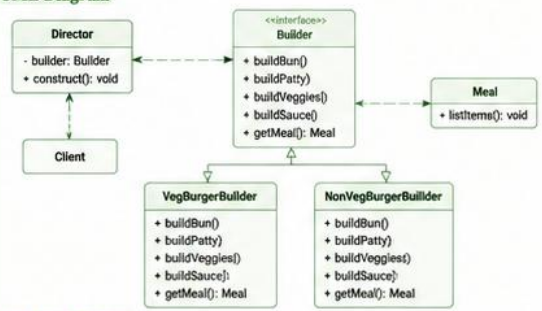
- When the construction process must allow different representations for the object that is constructed.
- When a complex object should be created step by step.

Example



A meal can be a veg or non-veg burger. We build the burger step by step using the same process but with different ingredients.

UML Diagram



Java Code Example

```
// Product
class Meal {
    private List<String> items = new ArrayList<>();
    public void addItem(String item) { items.add(item); }
    public void listItems() {
        System.out.println("Meal contains: " + items);
    }
}

// Builder
interface Builder {
    void buildBun();
    void buildPatty();
    void buildVeggies();
    void buildSauce();
    Meal getMeal();
}
// Concrete Builder (Veg)
class VegBurgerBuilder implements Builder {
    private Meal meal = new Meal();
    public void buildBun() { meal.addItem("Veg Bun"); }
    public void buildPatty() { meal.addItem("Veg Patty"); }
    public void buildVeggies() { meal.addItem("Lettuce, Tomato"); }
    public void buildSauce() { meal.addItem("Mayonnaise"); }
    public Meal getMeal() { return meal; }
}
```

```
// Concrete Builder (Non-Veg)
class NonVegBurgerBuilder implements Builder {
    private Meal meal = new Meal();
    public void buildBun() { meal.addItem("Non-Veg Bun"); }
    public void buildPatty() { meal.addItem("Chicken Patty"); }
    public void buildVeggies() { meal.addItem("Lettuce, Tomato"); }
    public void buildSauce() { meal.addItem("Tomato Sauce"); }
    public Meal getMeal() { return meal; }
}

// Director
class Director {
    private Builder builder;
    public Director(Builder builder) {
        this.builder = builder;
    }
    public void construct() {
        builder.buildBun();
        builder.buildPatty();
        builder.buildVeggies();
        builder.buildSauce();
    }
}
```

How it Works



Key Benefits

- ✓ Same construction process for different representations.
- ✓ Better control over complex object creation.
- ✓ Follows Single Responsibility Principle.

Trade-off

- More classes and code to implement.

3. PROTOTYPE PATTERN

Concept: Specifies the kinds of objects to create using a prototypical instance, and creates new objects by copying this prototype.

When to Use

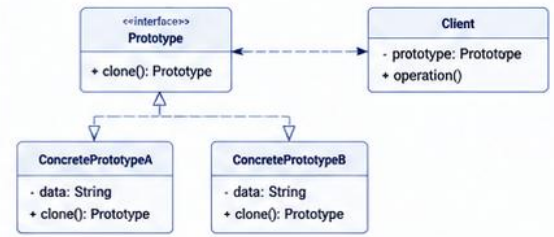
- When the classes to instantiate are specified at runtime.
- To avoid building a class hierarchy of factories that parallels the class hierarchy of products.

Example



A document editor can clone existing documents (letter, report) to create new ones with the same content.

UML Diagram



Java Code Example

```
// Prototype Interface
interface Document extends Cloneable {
    Document clone();
    void showData();
}
// Concrete Prototype A
class Letter implements Document {
    private String content;
    public Letter(String content) { this.content = content; }
    public Document clone() {
        return new Letter(this.content);
    }
    public void showData() {
        System.out.println("Letter: " + content);
    }
}
```

```
// Concrete Prototype B
class Report implements Document {
    private String content;
    public Report(String content) { this.content = content; }
    public Document clone() {
        return new Report(this.content);
    }
    public void showData() {
        System.out.println("Report: " + content);
    }
}

// Client
class Client {
    public static void main(String[] args) {
        Document letter = new Letter("Hello, this is a letter.");
        Document letterCopy = letter.clone();

        Document report = new Report("Quarterly Report Data...");
        Document reportCopy = report.clone();

        letter.showData();
        letterCopy.showData();
        reportCopy.showData();
    }
}
```

How it Works



Key Benefits

- ✓ Reduces subclassing.
- ✓ Creates objects at runtime.
- ✓ Performance improvement (cloning is faster).

Trade-off

- Cloning complex objects can be tricky.

Structural Patterns

- ✓ Describe how objects and classes are organized and connected.
- ✓ Support decomposition and flexible relationships.
- ✓ Examples: Facade, Adapter, Composite, Proxy, Decorator.

Facade Pattern

- Provides a simple interface to a complex subsystem.
- Hides internal subsystem details from clients.
- Reduces coupling and simplifies interaction.

Provides a simplified interface to a complex subsystem.
It hides internal subsystem details from clients.

Facade Pattern Steps

1. Design subsystem interfaces.
2. Implement subsystem classes.
3. Create facade wrapper class.
4. Client uses facade instead of multiple subsystem classes.

Adapter Pattern

- Allows incompatible interfaces to communicate.
- Works as a bridge between client and existing system.
- Components: Client, Target Interface, Adapter, Adaptee.

Adapter Pattern Steps

1. Design target interface.
2. Implement adapter using target interface.
3. Convert client requests into adaptee format.
4. Client communicates through adapter.

Composite Pattern

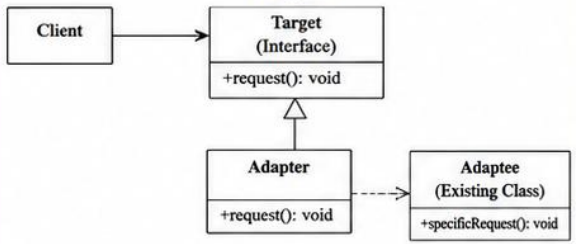
- Creates tree-like object structures.
- Allows individual objects and groups to be treated uniformly.
- Examples: file systems, organization structures.

Structural Design Patterns: Adapter, Bridge, Composite

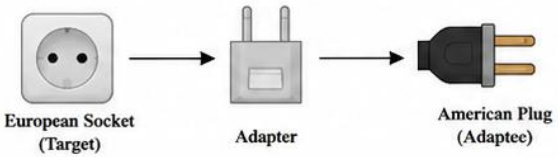
1. ADAPTER PATTERN

Adapter pattern allows incompatible interfaces to work together.

Diagram



Example: Power Plug Adapter



Java Code Example

```
// Target interface
interface EuropeanSocket {
    void providePower();
}

// Adaptee class
class AmericanPlug {
    public void provide110Volt() {
        System.out.println("Providing 110 Volts");
    }
}

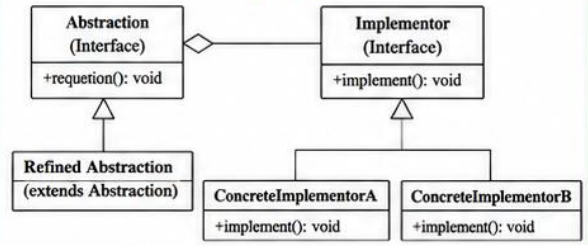
// Adapter class
class SocketAdapter implements EuropeanSocket {
    private AmericanPlug plug;
    public SocketAdapter(AmericanPlug plug) {
        this.plug = plug;
    }
    public void providePower() {
        plug.provide110Volt();
    }
}

// Client
EuropeanSocket socket = new SocketAdapter(new AmericanPlug());
socket.providePower();  // Output: Providing 110 Volts
```

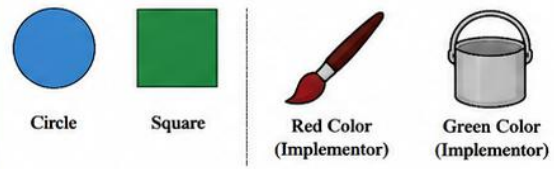
2. BRIDGE PATTERN

Bridge pattern decouples an abstraction from its implementation so that the two can vary independently.

Diagram



Example: Drawing Shapes



Java Code Example

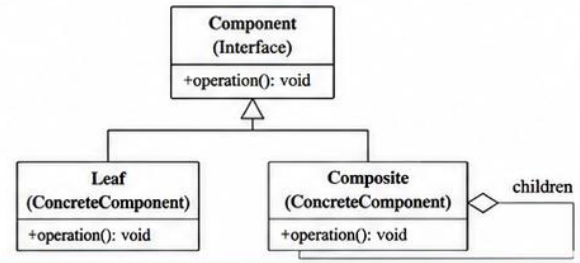
```
// Implementor
interface Color {
    void applyColor();
}
class RedColor implements Color {
    public void applyColor() {
        System.out.println("Applying Red Color");
    }
}
class GreenColor implements Color {
    public void applyColor() {
        System.out.println("Applying Green Color");
    }
}
// Abstraction
abstract class Shape {
    protected Color color;
    public Shape(Color color) { this.color = color; }
    abstract void draw();
}
class Circle extends Shape {
    public Circle(Color color) { super(color); }
    public void draw() {
        System.out.print("Drawing Circle -> ");
        color.applyColor();
    }
}

// Client
Shape s1 = new Circle(new RedColor());  s1.draw();
Shape s2 = new Circle(new GreenColor()); s2.draw();
```

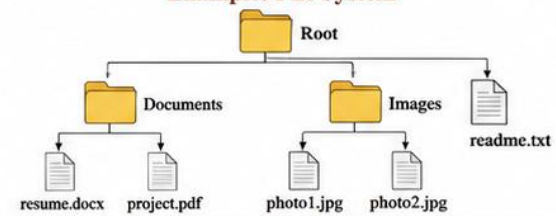
3. COMPOSITE PATTERN

Composite pattern composes objects into tree structures to represent part-whole hierarchies.

Diagram



Example: File System



Java Code Example

```
// Component
interface FileSystemComponent {
    void showDetails();
}
// Leaf
class File implements FileSystemComponent {
    private String name;
    public File(String name) { this.name = name; }
    public void showDetails() {
        System.out.println("File: " + name);
    }
}

// Composite
class Folder implements FileSystemComponent {
    private String name;
    private List<FileSystemComponent> children = new ArrayList<>();
    public Folder(String name) { this.name = name; }
    public void add(FileSystemComponent comp) { children.add(comp); }
    public void showDetails() {
        System.out.println("Folder: " + name);
        for (FileSystemComponent comp : children) {
            comp.showDetails();
        }
    }
}
// Client
Folder root = new Folder("Root");
Folder docs = new Folder("Documents");
docs.add(new File("resume.docx"));
docs.add(new File("project.pdf"));
root.add(docs);
root.add(new Folder("Images"));
root.add(new File("readme.txt"));
root.showDetails();
```

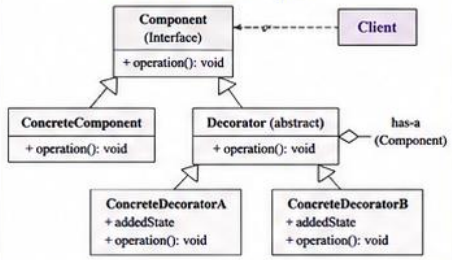
Structural Design Patterns: Decorator, Facade, Flyweight, Proxy (Server)

These patterns organize classes and objects to form larger structures while keeping the system flexible and easier to maintain.

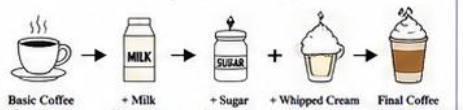
1. DECORATOR PATTERN

Decorator pattern attaches additional responsibilities to an object dynamically. It provides a flexible alternative to subclassing for extending functionality.

Structure / UML Diagram



Example: Coffee Shop



Java Code Example (Simplified)

```
// Component
interface Coffee {
    String cost();
    String description();
}

// Concrete Component
class SimpleCoffee implements Coffee {
    public String cost() { return "50"; }
    public String description() { return "Simple Coffee"; }
}

// Decorator (abstract)
abstract class CoffeeDecorator implements Coffee {
    protected Coffee coffee;
    public CoffeeDecorator(Coffee c) { this.coffee = c; }
}

// Concrete Decorator
class MilkDecorator extends CoffeeDecorator {
    public MilkDecorator(Coffee c) { super(c); }
    public String cost() { return coffee.cost() + " + 10"; }
    public String description() { return coffee.description() + ", Milk"; }
}
```

When to Use

- When you want to add responsibilities to objects dynamically.
- When subclassing leads to too many classes.
- When the extension should be flexible at runtime.

Key Benefits

- More flexible than inheritance.
- Allows responsibilities to be added/removed at runtime.
- Follows Open/Closed Principle.
- Combinations can be created easily.

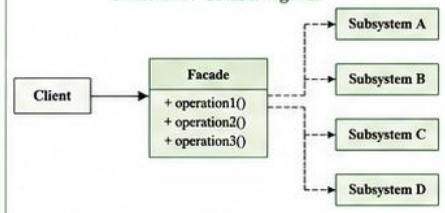
Think & Discuss

What new decorators would you add to the Coffee Shop example? Try adding VanillaDecorator and ChocolateDecorator.

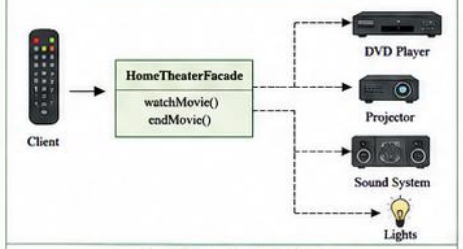
2. FACADE PATTERN

Facade pattern provides a simplified interface to a complex subsystem. It hides the complexities and makes the subsystem easier to use.

Structure / UML Diagram



Example: Home Theater System



Java Code Example (Simplified)

```
class HomeTheaterFacade {
    private DVDPlayer dvd;
    private Projector projector;
    private SoundSystem sound;
    private Lights lights;

    public HomeTheaterFacade(DVDPlayer dvd, Projector projector,
            SoundSystem sound, Lights lights) {
        this.dvd = dvd; this.projector = projector;
        this.sound = sound; this.lights = lights;
    }
    public void watchMovie() {
        lights.dim();
        projector.on();
        dvd.play();
        sound.on();
    }
    public void endMovie() {
        dvd.stop();
        projector.off();
        sound.off();
        lights.on();
    }
}
```

When to Use

- When a complex subsystem is difficult to use.
- When you need a simple entry point to use the system.
- When you want to decouple clients from subsystem classes.

Key Benefits

- Reduces complexity for clients.
- Promotes loose coupling.
- Organizes subsystem classes.
- Easier to understand and use.

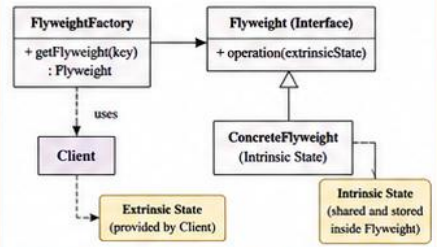
Think & Discuss

Why should the client not directly use the subsystem classes? What happens if the subsystem changes?

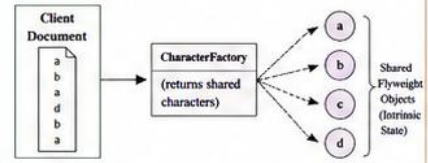
3. FLYWEIGHT PATTERN

Flyweight pattern uses sharing to support large numbers of fine-grained objects efficiently. It reduces memory usage by sharing common state among objects.

Structure / UML Diagram



Example: Text Editor (Characters)



Java Code Example (Simplified)

```
interface Character {
    void display(int position);
}

class ConcreteCharacter implements Character {
    private char symbol; // intrinsic state (shared)
    public ConcreteCharacter(char c) { this.symbol = c; }
    public void display(int position) {
        System.out.println("'" + symbol + "' at " + position);
    }
}

class CharacterFactory {
    private Map<Character, Character> map = new HashMap<>();
    public Character getCharacter(char c) {
        return map.computeIfAbsent(c, k -> new ConcreteCharacter(k));
    }
}
```

When to Use

- When your system has a large number of similar objects.
- When most of the object state can be made intrinsic (shared).
- When memory usage is critical.

Key Benefits

- Saves memory.
- Improves performance.
- Scalable with large numbers of objects.

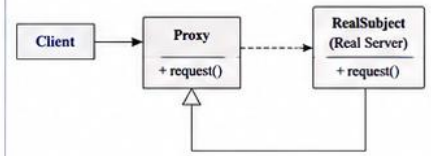
Think & Discuss

Where can you apply Flyweight Pattern in real life? (E.g., game development, text editors, icons, etc.)

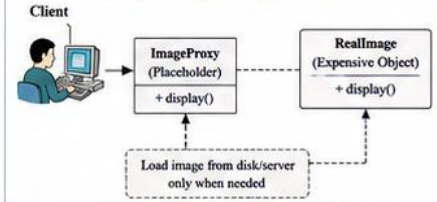
4. PROXY (SERVER) PATTERN

Proxy pattern provides a surrogate or placeholder for another object to control access to it. The proxy controls access to the real object (server).

Structure / UML Diagram



Example: Image Proxy (Server)



Java Code Example (Simplified)

```
interface Image {
    void display();
}
class RealImage implements Image {
    private String fileName;
    public RealImage(String fileName) {
        this.fileName = fileName;
        loadFromDisk();
    }
    private void loadFromDisk() {
        System.out.println("Loading " + fileName + " ...");
    }
    public void display() {
        System.out.println("Displaying " + fileName);
    }
}
class ImageProxy implements Image {
    private RealImage realImage;
    private String fileName;
    public ImageProxy(String fileName) { this.fileName = fileName; }
    public void display() {
        if (realImage == null) realImage = new RealImage(fileName);
        realImage.display();
    }
}
```

When to Use

- When object creation is expensive.
- When you need access control.
- When you need lazy loading.
- When you need remote proxies (e.g., server communication).

Key Benefits

- Controls access to objects.
- Allows lazy initialization.
- Reduces cost of object creation.
- Improves performance.

Think & Discuss

How can Proxy Pattern be used in a real-world application like a caching server or firewall?

Quick Comparison

- Decorator adds behavior dynamically by wrapping objects.
- Facade provides a simple interface to a complex subsystem.
- Flyweight shares common objects to save memory.
- Proxy controls access to another object and manages its creation.

Proxy Pattern

- ✓ Provides a substitute object controlling access to another object.
- ✓ Used for security, performance optimization, and resource management.

Decorator Pattern

- ✓ Dynamically adds responsibilities to objects at runtime.
- ✓ Uses aggregation instead of creating many subclasses.
- ✓ Allows flexible combinations of behaviours.

Decorator Pattern Steps

1. Design component interface.
2. Implement concrete component.
3. Implement abstract decorator.
4. Create concrete decorators adding behaviours.

Module 1 Summary

- Design patterns provide reusable solutions for common design problems.
- Creational patterns manage object creation.
- Structural patterns organize object relationships.
- Good pattern selection improves maintainability and flexibility.

Sample Question

Bloom's Level	Focus	Typical Question Verbs
Remember	Recall facts/concepts	Define, List, Name, Identify
Understand	Explain concepts	Explain, Describe, Summarize, Discuss
Apply	Use knowledge in a situation	Apply, Demonstrate, Implement, Use
Analyze	Break down and find relationships/problems	Analyze, Compare, Differentiate, Examine
Evaluate	Make a justified judgment	Evaluate, Critique, Justify, Assess
Create	Produce a new design/solution	Design, Develop, Create, Propose

Bloom's Level	Question
Remember	1. What is a design pattern? Name the three major categories of GoF design patterns.
Understand	2. Explain the purpose of the Gang of Four (GoF) design pattern catalog.
Understand	3. Explain the difference between creational and structural design patterns with suitable examples.
Apply	4. Suppose an application requires exactly one shared configuration object throughout the system. Which design pattern would you use? Explain how you would apply it.
Analyze	5. Analyze the following situation: a program directly creates many different concrete objects using new statements throughout the client code. Identify the design problem and determine whether the Factory Method Pattern could improve the design.
Evaluate	6. A developer uses the Singleton Pattern for every shared object in an application. Evaluate this design decision and discuss its advantages and possible disadvantages.
Create	7. Design a simple Factory Method-based system for creating different types of university courses such as TheoryCourse, LabCourse, and ProjectCourse. Describe the classes and their relationships.

Bloom's Level	Question
Remember	1. List the seven structural/creational patterns covered in Module 1.
Understand	2. Explain the main purpose of the Facade Pattern and give one practical example.
Understand	3. Explain how the Adapter Pattern allows two incompatible interfaces to work together.
Apply	4. A legacy payment system provides makePayment(), while a new application expects pay(). Which pattern would you apply and how would it solve the problem?
Analyze	5. Analyze a file-system design containing File and Folder objects. Explain why the Composite Pattern may be appropriate for representing this hierarchy.
Evaluate	6. Compare the Proxy and Facade Patterns. Evaluate which pattern would be more appropriate for controlling access to an expensive remote service and justify your answer.
Create	7. Design a home-theater system using the Facade Pattern. Identify the subsystem classes, Facade class, and the operations exposed to the client.

Bloom's Level	Question
Remember	1. Define Singleton, Factory Method, Adapter, Facade, Composite, Proxy, and Decorator Patterns.
Understand	2. Explain how the Decorator Pattern adds responsibilities to an object dynamically.
Understand	3. Explain the difference between inheritance and aggregation in the context of the Decorator Pattern.
Apply	4. A coffee application needs to dynamically add milk, sugar, and whipped cream to a basic coffee object. Which design pattern would you use? Explain the implementation approach.
Analyze	5. Analyze a system in which separate subclasses have been created for every possible combination of features, such as CoffeeWithMilk, CoffeeWithSugar, CoffeeWithMilkAndSugar, and CoffeeWithMilkSugarCream. Identify the design problem and suggest a suitable pattern.
Evaluate	6. Evaluate the use of the Decorator Pattern compared with creating multiple subclasses for every combination of additional functionality.
Create	7. Design a Java-based notification system using the Decorator Pattern in which basic notifications can dynamically be enhanced with email, SMS, and push-notification features.

Sample Question

Bloom's Level	Question
Remember	1. What is the main purpose of creational design patterns? Name two creational patterns discussed in Module 1.
Understand	2. Explain how the Singleton Pattern controls object creation.
Understand	3. Explain the difference between the Facade Pattern and the Adapter Pattern.
Apply	4. A university system requires a single object to manage application-wide configuration settings. Which pattern would you select? Explain how you would implement it.
Analyze	5. Analyze a system in which the client directly creates and manages several objects belonging to a complex subsystem. Identify the design problem and explain how the Facade Pattern could simplify the design.
Evaluate	6. Evaluate whether the Adapter Pattern or Facade Pattern is more appropriate when two existing classes have incompatible interfaces. Justify your answer.
Create	7. Design a Java-based file-management system using the Composite Pattern. Show how individual files and folders can be treated uniformly.

Bloom's Level	Question
Remember	1. What are the three major categories of the GoF design patterns? Give two examples from each category.
Understand	2. Explain the role of the Factory Method Pattern in separating object creation from object usage.
Understand	3. Explain how a Proxy object controls access to a Real Subject.
Apply	4. A web application loads very large images, but the images should only be loaded when they are actually displayed. Which design pattern would you apply? Explain your approach.
Analyze	5. Analyze a design where a program creates a separate subclass for every possible combination of additional features. Explain why this design may become difficult to maintain and identify a suitable pattern.
Evaluate	6. Compare inheritance-based extension with the Decorator Pattern. Evaluate which approach is more suitable when responsibilities need to be added dynamically at runtime.
Create	7. Create a Java notification system in which a basic notification can dynamically be extended with SMS, email, and push-notification capabilities using the Decorator Pattern.

Bloom's Level	Question
Remember	1. Define the Adapter, Composite, Proxy, and Decorator Patterns.
Understand	2. Explain the purpose of Pattern Languages and how multiple design patterns can work together.
Understand	3. Explain why design patterns are considered reusable design solutions rather than ready-made programs.
Apply	4. An application needs to communicate with an old payment service whose interface is incompatible with the application's current payment interface. Select and apply an appropriate design pattern.
Analyze	5. Analyze the following situation: a client directly communicates with five subsystem classes to perform one high-level operation. Identify the design problem and explain how the Facade Pattern can improve it.
Evaluate	6. Evaluate the following statement: **"The Singleton Pattern should be used whenever an object is accessed from multiple parts of a program."** Give reasons for your answer.
Create	7. Design an e-commerce system that uses at least **two appropriate design patterns** from Module 1. Explain why you selected each pattern and describe how the patterns cooperate.